Research on Service Composition Based on DQN in Multiple Clouds

Yanhua Yang*
College of Computer Engineering
Jimei University
Xiamen, Fujian, China
yhyang@jmu.edu.cn

Xinrong Zhan
School of Data and Computer Science
Xiamen Institute of Technology
Xiamen, Fujian, China
zhanxinrong@xit.edu.cn

Abstract

Service organized all kinds of services to one for the user. In the cloud environment, the services and related QoSs (Quality of Services) in every cloud may be different. In this paper, how to compose those services together in the multiple clouds environment is the problem. We use a SCDQN (Service Composition based a Deep-Q-Network) method, and consider the execution time, privacy level, cost and availability of the service and the service request in the multiple clouds. Simulations show that SCDQN has a lower value in the execution time and cost while satisfying privacy level and availability requirement.

CCS Concepts

• **Information systems** → World Wide Web; Web services.

Keywords

Service composition, Multiple clouds, Execution time, DQN

ACM Reference Format:

Yanhua Yang and Xinrong Zhan. 2024. Research on Service Composition Based on DQN in Multiple Clouds. In *2024 5th International Conference on Computer Science and Management Technology (ICCSMT 2024), October 18–20, 2024, Xiamen, China.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3708036.3708063

1 Introduction

Service composition has become an essential approach in numerous domains, enabling the integration of diverse services into a cohesive workflow [1]. This method is particularly useful in scenarios where individual services are not sufficient to accomplish complex tasks, and combining them allows for enhanced functionality and efficiency[2].

In cloud computing environments, service composition presents several challenges, particularly concerning resource allocation, quality of service (QoS), and scalability [3,4]. First, the dynamic nature of cloud environments, where resources can be scaled up or down depending on demand, complicates the task of ensuring that service compositions remain efficient and reliable. The variability of cloud resources in terms of performance, cost, and availability also introduces uncertainty in the selection process. Another challenge is meeting QoS requirements such as latency, throughput, and availability, which must be maintained while orchestrating multiple services across different cloud platforms. Several approaches have been proposed to address the challenges of service composition. Classical optimization methods, such as linear programming and heuristic-based methods, have been employed to optimize the selection and orchestration of services [5–7]. However, these methods often struggle to cope with the complexity and scalability of modern cloud environments, especially when dealing with a large number of services and diverse QoS constraints. Deep Q-Networks (DQN) have been widely applied to solve complex optimization problems across various fields. And we try to solve the service composition problem under multiple clouds.

2 Related work

Service composition has widely researched due to it can integrate multiple independent services into a unified workflow, enabling complex functionality that cannot be achieved by a single service [1,8]. The primary targets of service composition include optimizing Quality of Service (QoS) attributes such as latency, throughput, availability, and security. However, service composition in cloud environments presents several challenges.

Numerous methods have been proposed to address the challenges of service composition[1,2,9,10]. Traditional approaches rely on heuristic-based and rule-based systems that focus on predefined selection criteria. While these methods are useful in static or moderately dynamic environments, they often struggle to adapt to highly dynamic cloud environments where service availability, performance, and costs fluctuate. More advanced techniques, such as mathematical optimization methods like linear programming and integer programming, have also been used to optimize service selection based on multiple QoS criteria. However, these approaches may suffer from scalability issues as the number of available services increases. Evolutionary algorithms, including Genetic Algorithms (GA)[11], Particle Swarm Optimization (PSO) [6], and Ant Colony Optimization ([1,12]), have emerged as more flexible solutions that can explore a larger solution space and handle multi-objective optimization problems. These algorithms have been particularly effective in dynamic environments, but they still face challenges in convergence speed and solution quality. Recent advancements have seen the application of machine learning, particularly reinforcement learning (RL), to the problem of service composition. These methods allow for adaptive decision-making,

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICCSMT 2024, October 18–20, 2024, Xiamen, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0999-9/24/10
https://doi.org/10.1145/3708036.3708063

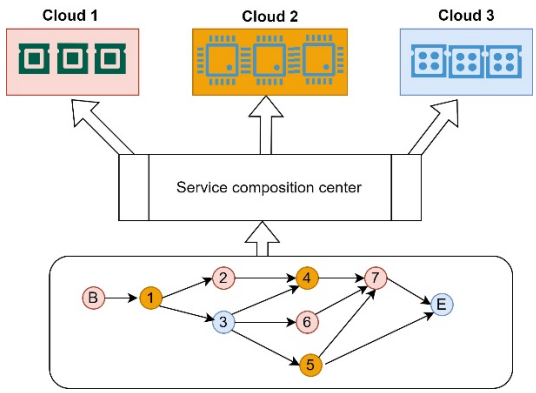

Figure 1: System model.

where the system can learn from its environment and optimize service selection in real-time.

Deep Q-Networks (DQN) is a reinforcement learning technique that has been widely used in recent years because of its efficiency in solving complex problems [7,13]. The advantage of using DQN for service composition lies in its adaptability and ability to optimize decision-making processes in dynamic environments [14].

3 System model

3.1 System framework

Figure 1 depicts the structure of a multi-cloud service composition system, which consists of three primary components: service providers in distinct cloud environments (Cloud 1, Cloud 2, and Cloud 3), a service composition center, and a directed acyclic graph (DAG) representing the service structure. At the core of the system is the service composition center, which is responsible for coordinating services across multiple clouds. The bottom section of the diagram illustrates a DAG model representing the flow of tasks from a beginning node 'B' to an end node 'E'. The nodes in the DAG represent distinct services, while the arrows depict the dependencies and execution flow between these services. The services in the DAG are scheduled on different cloud environments—Cloud 1, Cloud 2, and Cloud 3. Each cloud is depicted with unique characteristics. The colored nodes in the DAG correspond to services that are mapped to specific cloud providers. For example, node 1 (orange) is best executed on Cloud 2 (orange), while node 3 (blue) is optimally placed on Cloud 3 (blue).

3.2 Models used in the system

In a multi-cloud environment, services often need to be orchestrated in a structured manner, especially when complex applications are broken into smaller dependent tasks. A Directed Acyclic Graph (DAG) is a suitable model for representing such service workflows, where each node represents a task or service, and the directed edges represent dependencies between these tasks. In this model, the overall execution considers key metrics such as execution time, privacy level, cost, and availability. A DAG of a service is defined:

$$G = (V, E) \tag{1}$$

where:

- $(V = \{v_1, v_2, \ldots, v_n\})$ denotes a set of nodes representing service components.

- $(E = \{(v_i, v_j) \mid v_i \rightarrow v_j\})$ is a set of directed edges representing dependencies between services.

Execution time $\mathrm{T(v_i)}$ represents the duration required to execute a service (v_i) in a cloud. The total execution time of the DAG depends on the critical path, which is the longest sequence of dependent tasks. The execution time of a service (v_i) on a cloud(c_j) can be expressed as:

$$T\left(v_i, c_j\right) = \frac{w_i}{P\left(c_j\right)} + \delta\left(v_i, c_j\right) \tag{2}$$

Where:

-$(w_i)$is the computational workload of service (v_i) (e.g., in millions of instructions or floating-point operations).

-$(P(c_j))$ is the processing power of cloud(c_j) (e.g., in FLOPS or MIPS).

-$(\delta(v_i, c_j))$ is the data transfer or communication delay if service (v_i) depends on data from other tasks (all predecessor tasks) executed on different clouds.

Each task (v_i) may have a specific privacy requirement $(L_p(v_i))$, defining the sensitivity of data processed by the service. Privacy levels are categorized into discrete levels (we give 5 levels in the simulation). Let $(R_p(c_j))$ denote the privacy protection provided by cloud (c_j). The privacy constraint is satisfied if:

$$L_p\left(v_i\right) \leq R_p\left(c_j\right) \tag{3}$$

This ensures that the cloud executing service (v_i) offers sufficient privacy protection based on the service privacy requirement.

The cost of executing a service in the cloud consists of computation cost and data transfer cost. The cost of executing a task (v_i) on cloud (c_j) can be expressed as:

$$C\left(v_i, c_j\right) = C_{comp}\left(v_i, c_j\right) + C_{trans}\left(v_i, c_j\right) \tag{4}$$

Where:

- $(C_{comp}(v_i, c_j))$ is the cost of computation (e.g., per CPU cycle or time unit).

- $(C_{trans}(v_i, c_j))$ is the cost of transferring input/output data to/from cloud (c_j).

The overall cost of the DAG execution is the sum of the costs for all individual tasks:

$$C_{total} = \sum_{v_i \in V} C\left(v_i, c_j\right) \tag{5}$$

Availability represents the reliability of a cloud service in executing a service request. Each cloud (c_j) provides a certain availability level$(A(c_j))$, typically expressed as a percentage (e.g., 99.9% availability).

The availability of task (v_i) on cloud (c_j) can be defined as:

$$A\left(v_i, c_j\right) = A\left(c_j\right) \tag{6}$$

The overall availability of the DAG is calculated by considering the availability of the critical path (in other words, the path that defines the overall execution time).

3.3 Service composition targets

Our objective is to minimize the overall execution time (Formula (7)), cost (Formula (8)), and meet privacy satisfaction and availability of the service composition in the DAG. The service composition can be expressed as the following problem:

$$min\ T_{total} = \max\left(\sum_{v_i \in \text{critical path}} T\left(v_i, c_j\right)\right) \tag{7}$$

$$min\ C_{total} = \sum_{v_i \in V} C\left(v_i, c_j\right) \tag{8}$$

where,

$$\forall i : L_p\left(v_i\right) \leq R_p\left(c_j\right) \tag{9}$$

$$A_G \leq \prod_{v_i \in \text{critical path}} A\left(v_i, c_j\right) \tag{10}$$

Where Formula (9) ensures the privacy constraint is satisfied, and 0 otherwise, Formula (10) ensures that the most crucial tasks are executed in a satisfied available environment.

4 Service composition method based on DQN

4.1 State Representation

In DQN, the state represents the current configuration of the task allocation in the DAG. A state (s) can be defined as a vector containing the following components: (1) Task status: Indicates whether each task ($v_i \in V$) is completed, in progress, or not yet start; (2) Cloud assignments: Which cloud (c_j) each task (v_i) is currently assigned to (if any); (3) Available cloud resources: Current resource availability (e.g., CPU, memory, bandwidth) of each cloud(c_j); (3) Remaining execution time for tasks currently being processed;(5)Data transfer status: Whether data dependencies between tasks (edges in the DAG) have been transferred and are ready for execution. The state at time (t) is represented as:

$$s_t = \{s\left(v_1\right), s\left(v_2\right), \ldots, s\left(v_n\right)\} \tag{11}$$

Where ($s(v_i)$) includes the task's execution status, the cloud it's assigned to, and resource consumption.

4.2 Action Space

The action in DQN corresponds to scheduling decisions, such as assigning tasks to available cloud resources. At each decision point, the agent can: (1) Select a task: Choose a task (v_i) that is ready for execution (i.e., all predecessor tasks are complete); (2) Assign a cloud: Assign the selected task to a specific cloud (c_j), ensuring that the resource, privacy, and availability requirements are met. The action (a_t) at time step (t) can be defined as:

$$a_t = \text{Assign}\left(v_i, c_j\right) \tag{12}$$

Where (v_i) is the selected task and (c_j) is the chosen cloud for execution.

4.3 Reward Function

(r_t) is the target of the scheduling problem. The agent receives a reward based on how well the selected action meets the desired goals:

$$r_t = \left(w_1 \cdot R_T\left(v_i, c_j\right) + w_2 \cdot R_C\left(v_i, c_j\right)\right) * \cdot R_P\left(v_i, c_j\right) * R_A\left(v_i, c_j\right) \tag{13}$$

Where: $R_T(v_i, c_j)$ is the e execution time penalty for task (v_i) on cloud (c_j), $R_C(v_i, c_j)$ is the cost penalty for computation and data transfer. For $R_P(v_i, c_j)$: if $\forall i : L_p(v_i) \leq R_p(c_j)$ (v_i is all completed tasks) returns 1, otherwise, it returns 0. For $R_A(v_i, c_j)$: if $A_G \leq \prod_{v_i \in \text{critical path}} A(v_i, c_j)$ (v_i is all completed tasks) returns 1, otherwise, it returns 0. The weights (w_1, w_2) are tunable parameters to balance the trade-offs between the two objectives.

4.4 Q-Value

The Q-value($Q(s_t, a_t)$) is the expected cumulative reward when taking action (a_t) from state (s_t) and taking the optimal policy. The DQN uses a neural network to approximate the Q-value:

$$Q\left(s_t, a_t\right) \approx Q^*\left(s_t, a_t; \theta\right) \tag{14}$$

Where (θ) is the parameters of the DQN. The input to the network is the state (s_t), and the output is a vector of Q-values corresponding to all possible actions.

4.5 Training the DQN

The DQN is trained using experience replay and the Q-learning update rule. The update equation for the Q-value is:

$$Q\left(s_t, a_t\right) \leftarrow Q\left(s_t, a_t\right) + \alpha\left[r_t + \gamma \max_{a'} Q\left(s_{t+1}, a'\right) - Q\left(s_t, a_t\right)\right] \tag{15}$$

Where: α, r_t and $\max_{a'} Q(s_{t+1}, a')$ are the Learning rate, discount factor for future rewards, and Maximum Q-value for the next state. Experience replays stores past transitions ((s_t, a_t, r_t, s_{t+1})) in a buffer and randomly samples them for training, reducing correlation between consecutive updates.

5 Simulation and composition

5.1 Simulation environment

In our simulation environment, we consider a heterogeneous multi-cloud system comprising 10 distinct cloud providers. Each cloud offers a diverse range of 200 different service types, characterized by variations in price (random in [1,5] in the simulation), execution time (random in [1,10] in the simulation), privacy level (random in [1,5] in the simulation), and availability (random in [0.6, 1] in the simulation). These parameters are essential for modeling the diverse quality of services in real-world cloud environments, and they are randomly assigned to reflect the heterogeneous nature of multiple clouds. For each cloud, the 200 services have an integer value assigned between [1, 50] based on a distributed distribution, accurately capturing the diversity and resource heterogeneity typically encountered in multi-cloud environments. To evaluate the service composition method, we generate service requests represented as Directed Acyclic Graphs (DAGs) that consist of between 100 and 700 interconnected services.

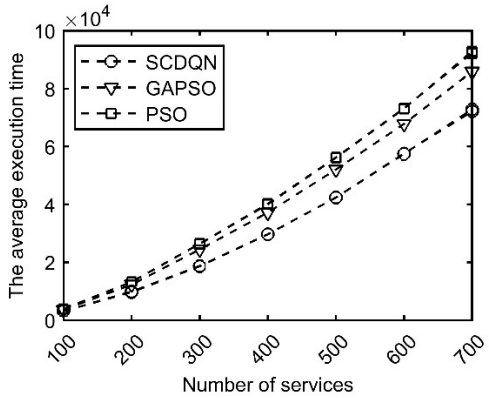

Figure 2: The average execution time.

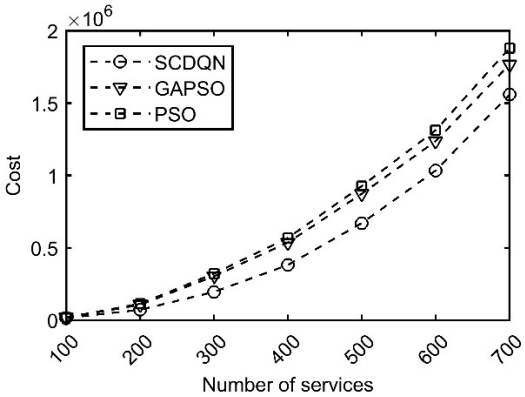

Figure 3: Cost of different methods.

5.2 Comparison and discussion

We will compare our method with PSO (particle swarm optimization) service composition method [5] and GA-PSO [15](a combination of genetic algorithm and particle swarm optimization algorithm).

In Figure 2, the x-axis and y-axis are the number of services and the average execution time (AET) in unspecified units, likely milliseconds or seconds. The trend for all three methods shows an increase in AET as the number of services rises. SCDQN maintains the lowest AET, followed by GA-PSO and PSO. At 700 services, SCDQN achieves an AET reduction of approximately 11.11% compared to GA-PSO and around 20% compared to PSO. These reductions underscore SCDQN's effectiveness in minimizing execution time. Similarly, in Figure 3, the x-axis and y-axis are the number of services and the cost, measured in unspecified units likely representing monetary or computational resource costs. The trend observed across all methods—SCDQN, GA-PSO, and PSO—indicates an increase in cost with the growing number of services. However, SCDQN consistently exhibits the lowest cost compared to GA-PSO and PSO. Specifically, at 700 services, SCDQN reduces the cost by approximately 15.79% compared to GA-PSO and by about 20% compared to PSO, highlighting its superior cost-efficiency.

The observed differences between the methods can be attributed to several factors. SCDQN's utilization of deep learning, specifically Deep Q-Networks, enhances its ability to learn and adapt to complex patterns within the data, enabling more informed decisions in service composition. This learning capability allows SCDQN to optimize service compositions more effectively than heuristic methods like PSO and GA-PSO.

6 Conclusion and future work

The DQN method offers a robust solution for solving the complex multi-objective service composition problem in multi-cloud environments. By learning an optimal task scheduling policy, it can effectively balance execution time, cost, privacy satisfaction, and availability, ensuring that service composition in DAG-based workflows is both efficient and compliant with system requirements.

Acknowledgments

Project funded by the Natural Science Foundation of Fujian Province, 2022J01820.

References

[1] Vakili A, Navimipour N J. Comprehensive and systematic review of the service composition mechanisms in the cloud environments[J]. Journal of Network and Computer Applications, Elsevier Ltd, 2017, 81(May 2016): 24–36.
[2] Asghari P, Rahmani A M, Javadi H H S. Service composition approaches in IoT: A systematic review[J]. Journal of Network and Computer Applications, Elsevier Ltd, 2018, 120(July): 61–77.
[3] Njima C Ben, Guegan C G, Gamha Y, *et al.* Web Service Composition in Mobile Environment: A Survey of Techniques[J]. IEEE Transactions on Services Computing, IEEE, 2024, 17(2): 689–704.
[4] Lin J, Huang S, Zhang H, *et al.* A Deep-Reinforcement-Learning-Based Computation Offloading with Mobile Vehicles in Vehicular Edge Computing[J]. IEEE Internet of Things Journal, IEEE, 2023, 10(17): 15501–15514.
[5] Haytamy S, Omara F. Enhanced QoS-Based Service Composition Approach in Multi-Cloud Environment[J]. Proceedings of 2020 International Conference on Innovative Trends in Communication and Computer Engineering, ITCE 2020, IEEE, 2020, 33–38.
[6] Tang C, Goh M, Zhao S, *et al.* Priority-based two-phase method for hierarchical service composition allocation in cloud manufacturing[J]. Computers and Industrial Engineering, Elsevier Ltd, 2024, 196(August): 110517.
[7] Boutarfa M, Maamri R, Lacheheub M N. Towards an approach for cloud service composition in Multi-Cloud environment based QoS using deep Q-learning ∗[J]. ICAASE 2022 - 5th Edition of the International Conference on Advanced Aspects of Software Engineering, Proceedings, IEEE, 2022, 1–7.
[8] Wang P, Xu J, Lei Y. Automated Web Service Composition Considering Service Intension and Extension[J]. Proceeding - 2021 China Automation Congress, CAC 2021, IEEE, 2021, 6549–6554.
[9] Huf A, Siqueira F. Composition of heterogeneous web services: A systematic review[J]. Journal of Network and Computer Applications, Elsevier Ltd, 2019, 143(August 2018): 89–110.
[10] Razian M, Fathian M, Bahsoon R, *et al.* Service composition in dynamic environments: A systematic review and future directions[J]. Journal of Systems and Software, Elsevier Inc., 2022, 188: 111290.
[11] Zhang Z, Zeng F. Efficient Task Allocation for Computation Offloading in Vehicular Edge Computing[J]. IEEE Internet of Things Journal, IEEE, 2023, 10(6): 5595–5606.
[12] Diao F, Jia Z, Wang R, *et al.* Cloud Service Composition and Optimization Selection Based on Hybrid Service Composition Algorithm[J]. Proceedings - 2022 8th Annual International Conference on Network and Information Systems for Computers, ICNISC 2022, 2022, 454–458.
[13] Qiu J, Wang R, Chakrabarti A, *et al.* Adaptive Edge Offloading for Image Classification Under Rate Limit[J]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, IEEE, 2022, 41(11): 3886–3897.
[14] Tang H, Wu H, Qu G, *et al.* Double Deep Q-Network Based Dynamic Framing Offloading in Vehicular Edge Computing[J]. IEEE Transactions on Network Science and Engineering, IEEE, 2023, 10(3): 1297–1310.
[15] Dogani J, Khunjush F. Cloud Service Composition Using Genetic Algorithm and Particle Swarm Optimization[J]. ICCKE 2021 - 11th International Conference on Computer Engineering and Knowledge, IEEE, 2021, (Iccke): 98–104.